LA TORRE DE MORVATH · CÓMO SE HIZO

De una imagen a un personaje jugable

El monstruo naranja: de un dibujo plano a un modelo 3D con esqueleto, 16 animaciones y su propio ataque, en la sala de equipo junto al Encapuchado, la Pícara, el Caballero y el Bárbaro.



La imagen de partida (una sola vista, de frente).

La idea en una frase

Una IA de «imagen a 3D» adivina la forma completa del monstruo, por delante y por detrás, a partir de la foto. Después, unos scripts propios le ponen **huesos** para que se pueda doblar, le dan **movimientos** con los mismos nombres que ya usa el juego y lo guardan como un archivo **GLB** que el juego carga igual que a los demás personajes.

La condición: nada de más de 0,5 GB de RAM

Los modelos de imagen a 3D que se pueden instalar en casa (TripoSR, Hunyuan3D, TRELLIS) necesitan varios GB de memoria y una tarjeta gráfica potente, y este portátil no tiene GPU. La solución fue **no instalar nada**:

- La IA se ejecuta en un **Space gratuito de Hugging Face** (TRELLIS, de Microsoft, en GPU compartida). Mi ordenador solo envía la imagen y recibe el modelo.
- El resto (convertir, montar el esqueleto, animar) son scripts de **Python con la biblioteca estándar y numpy**, que ya estaban instalados.
- Memoria usada en local: unas decenas de MB. Coste: 0 €. Descargas de programas: ninguna.

Los cinco pasos

1 · Generar
TRELLIS quita el fondo y crea la malla con textura (25 s).

2 · OBJ
Un conversor propio pasa el GLB a OBJ + MTL + PNG.

3 · Medir
Se corta la malla en rodajas para saber dónde están piernas, brazos y cuello.

4 · Huesos y animación
13 huesos, pesos de piel y 16 animaciones con nombres de KayKit.

5 · Al juego
Una línea en la lista de personajes, su ataque y su sonido. Pruebas y git push.

Paso 1 · Generar el modelo sin instalar nada

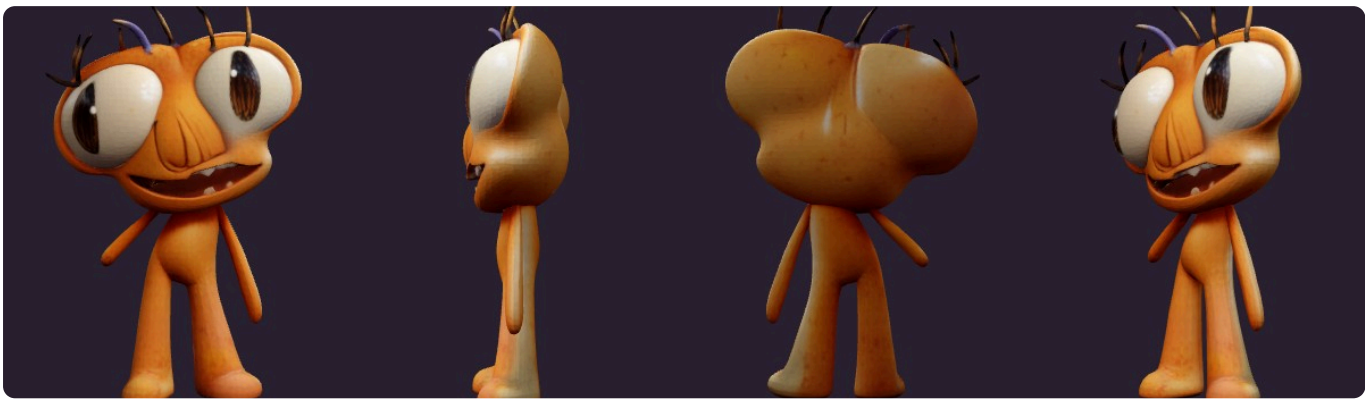
Los Spaces de Hugging Face tienen detrás una aplicación **Gradio**. Su página web habla con el servidor mediante una cola: se apunta uno a ella (`/queue/join`) y escucha los resultados por un flujo de eventos (`/queue/data`, SSE). El script `generar_3d.py` imita ese protocolo con `urllib`: abre una sesión, sube la imagen, pide *preprocess_image* (quita el fondo) y *generate_and_extract_glb* (forma + textura), y descarga el GLB.

```
python generar_3d.py monstruo_referencia.jpg monstruo_trellis.glb
Abriendo sesión… → Subiendo la imagen… → Quitando el fondo… → Generando forma + textura…
Listo en 25 s -> monstruo_trellis.glb   (1,6 MB: 9.993 vértices, 15.754 caras, textura 1024 px)
```

Primer tropiezo. Llamar a cada función por separado devolvía *«404: Session not found»*. TRELLIS guarda archivos por sesión, así que las tres llamadas tienen que ir en **la misma sesión de la cola**, como hace la web. Los números de función (`fn_index` 4, 8 y 11) salen de la configuración pública del Space (`/config`).

Paso 2 · El OBJ (y por qué el juego usa GLB)

Un GLB es glTF en binario: un bloque JSON que describe la escena y un bloque BIN con los números. `glb_a_obj.py` lee los dos, saca posiciones, coordenadas de textura (con la V invertida, porque glTF y OBJ la cuentan al revés) e índices, y escribe `monstruo.obj`, `monstruo.mtl` y `monstruo.png`. El OBJ es el formato universal, el que abre cualquier programa 3D. Pero **no admite huesos ni animaciones**, así que el juego carga un GLB.



El OBJ en un visor propio con Three.js: frente, lado, espalda y 3/4. La IA inventó la parte de atrás, que no salía en la imagen, de forma coherente.

Paso 3 · Medir el cuerpo

Para poner huesos hay que saber dónde están las articulaciones. `analizar.py` corta la malla en rodajas de 2,5 cm de alto y, en cada una, busca los huecos de más de 2 cm a lo largo del eje x. Así se distinguen las piezas separadas:

Altura (y)	Lo que se ve en la rodaja	Conclusión
−0,50 a −0,23	dos grupos: $x \in [-0{,}13;\ 0]$ y $x \in [0{,}04;\ 0{,}22]$	dos piernas; la rodilla, hacia −0,36
−0,33 a −0,08	grupos estrechos a los lados del tronco	dos brazos finos, de distinto largo
−0,22 a −0,05	un solo grupo de unos 18 cm	tronco; la cadera, en −0,215
−0,04 a 0,50	un grupo de más de 60 cm de ancho	la cabezota (7.275 de los 9.993 vértices)

Paso 4a · El esqueleto y los pesos de piel

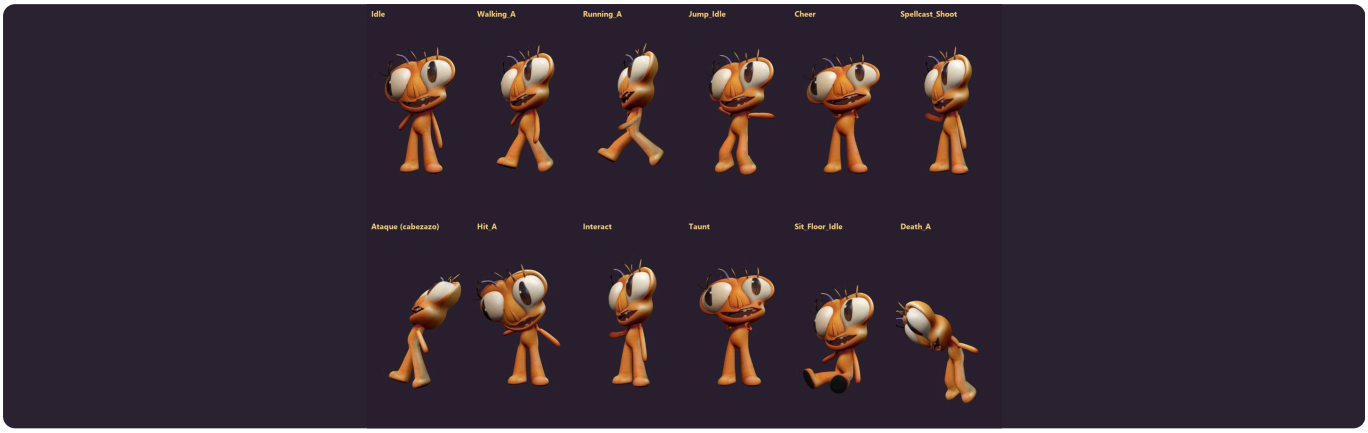
`construir_rig.py` crea **13 huesos**: raíz, cadera, tronco, cabeza, dos tramos por pierna y dos por brazo. En las manos añade los puntos `handslotr/handslotl`, que es donde el juego cuelga la varita. Luego decide qué hueso mueve cada vértice (**pesos de piel**):

- **Por regiones:** por encima del cuello, la cabeza; fuera del borde del tronco, los brazos; por debajo de la ingle, cada pierna según su lado.
- **Con fundido suave** en cada articulación (función *smoothstep*): cerca del codo, de la rodilla o del cuello, el vértice reparte su peso entre los dos huesos, para que la malla se doble en curva y no se quiebre.
- Cada vértice guarda sus 4 huesos principales con pesos que suman 1, que es lo que exige glTF.

> **Segundo tropiezo.** Las primeras capturas salían llenas de triángulos estirados. El cálculo en el procesador (`getVertexPosition`) daba aristas de 8 cm como mucho, así que el esqueleto estaba bien: el fallo era cómo hacía las fotos. Pintaba cuatro poses distintas en el mismo fotograma, y la GPU reutilizaba los huesos de la pose anterior. Pintando cada pose en su propio fotograma, todo salió limpio. En el juego nunca pasa, porque hay una pose por fotograma.

Paso 4b · Los movimientos

Cada animación es una función matemática del tiempo: para cada instante devuelve el giro de cada hueso (en grados, que luego pasan a cuaterniones) y un desplazamiento de la cadera para los botes. Se muestrea a 24 fotogramas por segundo y se guarda dentro del GLB. Al andar, por ejemplo, las piernas oscilan con seno y las rodillas solo se doblan hacia atrás; los brazos van a contratiempo y la cadera rebota dos veces por ciclo. Lo importante es que **se llaman igual que en los personajes de KayKit**, así que el jugador, los compañeros en equipo y las cinemáticas los usan sin tocar su código.



12 de las 16 animaciones en el visor. Además: Hit_B, Spellcasting, Dodge_Backward y Sit_Floor_StandUp.

> **Ajuste de diseño.** En la primera versión de *Cheer* (celebrar), los brazos subían 150° y quedaban tapados por la cabezota. Los brazos son cortos, así que ahora se abren en cruz (95°) y se agitan. Los modelos de IA traen sus propias proporciones, y las animaciones se adaptan a ellas, no al revés.

Paso 5 · Meterlo en el juego

Archivo	Cambio
`assets/modelos/monstruo.glb`	el modelo final: malla, textura JPEG, 13 huesos y 16 animaciones (0,95 MB)
`js/personajes.js`	una entrada nueva, *Monstruo*, sin paleta (trae su propia textura; los demás usan la paleta oscura común)
`js/armas.js` · `ataques.js` · `audio.js`	un quinto tipo de ataque, el **cabezazo mordedor**: corto (2,1 m) y rápido, con chispas naranjas y un «ñam» sintetizado
`herramientas/monstruo/`	los scripts, el OBJ, el visor y este documento, para poder repetirlo con otra imagen



Ya en el Piso II, como jugador (cámara a su espalda): de pie, celebrando, cabezazo y lanzando un hechizo.

Para repetirlo con otro dibujo

```
cd herramientas/monstruo
python generar_3d.py mi_imagen.jpg modelo.glb        # 1 · IA en Hugging Face (gratis)
python glb_a_obj.py modelo.glb modelo.obj            # 2 · OBJ universal
python analizar.py modelo.glb                        # 3 · medir: dónde poner los huesos
python construir_rig.py modelo.glb ../../assets/modelos/nuevo.glb   # 4 · huesos y animaciones
```

En el paso 4 hay que copiar las medidas del paso 3 en la tabla `HUESOS`: cada personaje tiene su cuerpo. Funciona mejor con imágenes de cuerpo entero, de frente, con brazos y piernas separados del tronco y sobre un fondo liso.

Herramientas: TRELLIS (Microsoft, MIT) en Hugging Face · Python 3.11 + numpy + Pillow · Three.js 0.170 · Chrome sin ventana para este PDF.